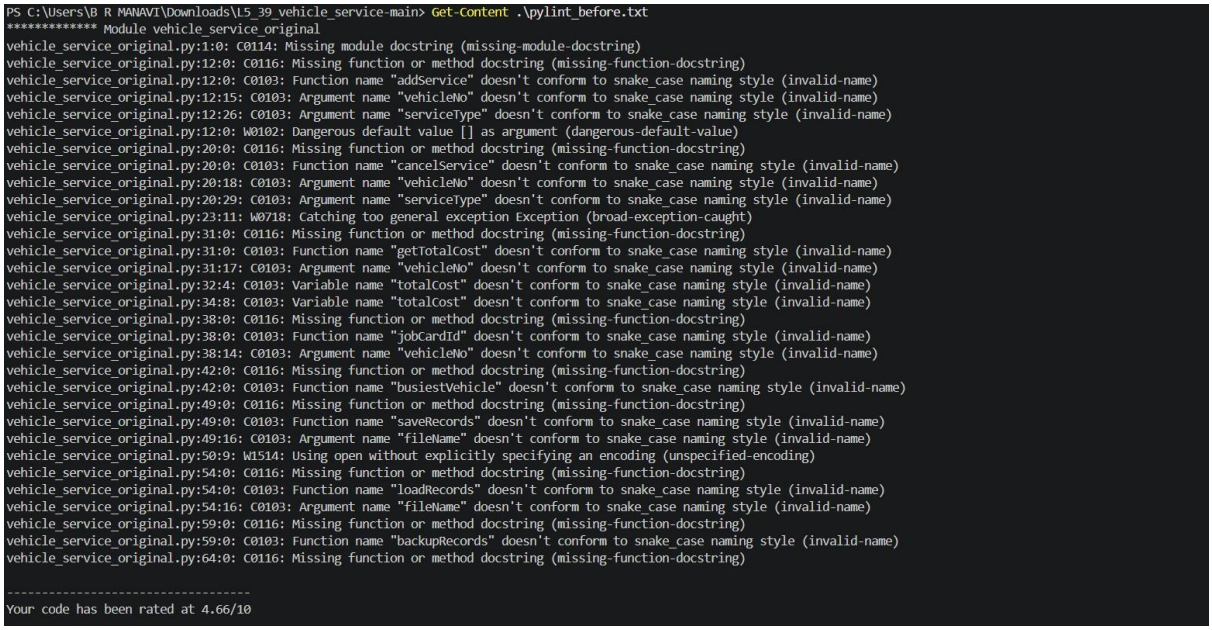


pylint before



bandit before

```
                                                    cmd /c ".venv\Scripts\python.exe -m bandit -r vehicle_service_original.py > bandit_before.txt 2>&1"
>> Get-Content .\bandit_before.txt5_39_vehicle_service-main>
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.14.0
Run started:2026-10-14 06:50:11.431810+00:00

Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess
   Location: .\vehicle_service_original.py:3:0
2       import json
3       import subprocess
4       from datetime import date

--------------------------------------------------
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'garage-demo'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: .\vehicle_service_original.py:8:17
7                       "General": 2500,"Brake Check": 1200}
8       ADMIN_PASSWORD = "garage-demo"
9       service_records = {}

--------------------------------------------------
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
   Location: .\vehicle_service_original.py:39:11
38      def jobCardId(vehicleNo):
39          return hashlib.md5(vehicleNo.encode()).hexdigest()[:8]
40

--------------------------------------------------
>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b602_subprocess_popen_with_shell_equals_true.html
   Location: .\vehicle_service_original.py:61:4
60          # Helper kept for the lab: not called by the demo
61          subprocess.call("cp services.json " + folder, shell=True)
62

--------------------------------------------------

Code scanned:
        Total lines of code: 59
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 2
                Medium: 0
                High: 2
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 3
Files skipped (0):
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main>
```

flake8 before

```
                High: 3
Files skipped (0):                                  cmd /c ".venv\Scripts\python.exe -m flake8 vehicle_service_original.py > flake8_before.txt 2>&1"
>> Get-Content .\flake8_before.txt5_39_vehicle_service-main>
vehicle_service_original.py:7:33: E231 missing whitespace after ','
vehicle_service_original.py:43:9: E225 missing whitespace around operator
vehicle_service_original.py:49:1: E302 expected 2 blank lines, found 1
vehicle_service_original.py:77:80: E501 line too long (83 > 79 characters)
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main>
```

Pylint after

```
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main> Get-Content .\pylint_report.txt

 ---------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Bandit after

```
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main> cmd /c ".venv\Scripts\python.exe -m bandit -r vehicle_service.py > bandit_report.txt 2>&1"
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main> Get-Content .\bandit_report.txt
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.14.0
Run started:2026-10-14 09:16:04.438311+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 110
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main>
```

flake8 after

```
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main> cmd /c ".venv\Scripts\python.exe -m flake8 vehicle_service.py > flake8_report.txt 2>&1"
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main> Get-Content .\flake8_report.txt
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main>
```

results

```
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main> .\.venv\Scripts\python.exe -m py_compile vehicle_service.py test_vehicle_service.py
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main> .\.venv\Scripts\python.exe -m unittest -v
test_add_service_cost_and_total (test_vehicle_service.VehicleServiceTests.test_add_service_cost_and_total) ... ok
test_cancel_existing_service (test_vehicle_service.VehicleServiceTests.test_cancel_existing_service) ... ok
test_cancel_missing_record_returns_false (test_vehicle_service.VehicleServiceTests.test_cancel_missing_record_returns_false) ... No service records found for vehicle KA09ZZ0001
ok
test_default_history_is_not_shared (test_vehicle_service.VehicleServiceTests.test_default_history_is_not_shared) ... ok
test_empty_vehicle_number_is_rejected (test_vehicle_service.VehicleServiceTests.test_empty_vehicle_number_is_rejected) ... ok
test_job_card_id_is_stable_and_short (test_vehicle_service.VehicleServiceTests.test_job_card_id_is_stable_and_short) ... ok
test_save_and_load_records (test_vehicle_service.VehicleServiceTests.test_save_and_load_records) ... ok
test_unknown_service_type_is_rejected (test_vehicle_service.VehicleServiceTests.test_unknown_service_type_is_rejected) ... ok

----------------------------------------------------------------------
Ran 8 tests in 0.020s

OK
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main>
```

final program

```
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main> .\.venv\Scripts\python.exe vehicle_service.py
WARNING: No service records found for vehicle KA09ZZ0001
Total for KA01AB1234: 2300
Total for KA05XY9876: 2500
Job card for KA01AB1234: bd2cd41f
Vehicles on file: 3 | busiest vehicle: KA01AB1234
PS C:\Users\B R MANAVI\Downloads\L5_39_vehicle_service-main>
```

SE
Lab 5: Static Code Analysis

Reflection Answers

Name: BR MANAVI	SRN: PES2UG24CS105
Lab: Static Code Analysis	Date: 14/10/2026

1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest issues to fix were the formatting and readability problems, such as using clearer names, improving the print statements, and organizing the code consistently. The harder fixes were the mutable default argument and the backup function. The default list could be shared between function calls, so I had to change how the history list was initialized. The backup function was harder because it used a shell command built from a string. I replaced that approach with Python file-handling tools and added checks for the source file and destination folder.

2. Did the static analysis tools report any false positives? If so, describe one example.

I did not identify a clear false positive in the findings I worked on. The issues were generally useful because they pointed to code that could be made safer or easier to maintain. For example, using a mutable default list can cause unexpected results across calls, and using shell=True with a command assembled from a string can be unsafe. I also learned that tool results should be reviewed instead of accepted blindly, because a warning still needs to be checked against what the program is doing.

3. How would you integrate static analysis tools into your actual software development workflow?

I would run Pylint, Bandit, and Flake8 after making code changes and before committing them to Git. I would keep the tools installed in the project virtual environment so the whole team can use the same setup. I would also add the checks to a GitHub Actions workflow so that the code is checked automatically when someone pushes changes or opens a pull request. This would help catch style problems and possible security issues earlier, before the code is merged.

4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

The code became easier to read because the function and variable names are clearer and the logic is more consistent. Input checks now reject an empty vehicle number and an unknown service type. The code no longer relies on a shared default list or runs a shell command for backups, and file operations specify UTF-8 where appropriate. I also added tests for the main behaviours. In my final checks, Python compilation succeeded, all 8 unit tests passed, Pylint gave a score of 10.00/10, Bandit reported no issues, and Flake8 produced no violations. These results gave me more confidence that the program still works after the changes.